

n-VM：具有共享身份与代币状态的多虚拟机Layer-1架构

Jian Sheng Wang

Yeah LLC

jason@yeah.app

arXiv:2603.23670v1 [cs.CR] 2026年3月24日

2026年3月23日

摘要

多链生态系统面临身份碎片化、流动性孤立以及依赖跨链桥进行代币转移等问题。我们提出 n-VM，一种在共享共识和共享状态上托管 n 个异构虚拟机作为对等执行环境的 Layer-1 架构。该设计融合了三个组件：一个依据操作码前缀路由交易的调度器、一个以单一32字节承诺锚定VM特定地址的统一身份层，以及一个在公共余额存储上暴露 ERC-20 和 SPL 等VM原生接口的统一代币账本。我们对路由、身份派生和代币转移语义进行了形式化定义，并在标准密码学假设下证明了跨VM转移的原子性与身份隔离性。我们描述了包含五种VM的具体实例：原生运行时、EVM、SVM、比特币脚本和TVM。此外，我们还介绍了基于上下文的分片机制和用于并行执行的写集调度器。在分析吞吐量模型下，该架构在普通硬件上的预测吞吐量约为16,000至66,000笔交易每秒。

关键词：多VM区块链、统一身份、跨VM互操作性、并行执行、上下文分片、操作码路由、ACE-GF

1 引言

1.1 动机

区块链生态系统在不兼容的执行环境中呈现碎片化状态。以太坊的EVM、Solana的SVM、比特币的Script以及Tron的TVM各自定义了不同的账户模型、地址格式、交易语义和智能合约语言。希望跨生态系统运营的用户必须维护独立的钱包、管理不同的密钥对，并依赖跨链桥——这些第三方系统历史上一直是去中心化金融中最大的单一安全失败来源，2021年至2024年间因桥接漏洞损失超过28亿美元[4]。

近期项目已开始探索多VM架构。Movement Labs在基于Move的结算链之上部署了EVM兼容的执行层；Eclipse将SVM执行环境作为以太坊Rollup运行；Sei v2在Cosmos SDK链中融合了EVM和CosmWasm。然而，这些方案共享一个共同局限：一个VM从属于另一个VM（作为Rollup或兼容层），各VM之间的身份依然碎片化，执行环境之间的代币转移仍然需要类似桥接的机制。

1.2 核心洞见：VM无关的身份-授权分离

Ace-GF框架[1]引入了身份绑定与每笔交易授权之间的分离机制。用户的链上身份由单一32字节承诺表示： $id_com = Poseidon(REV, salt, domain)$ ，通过HKDF密钥流从根熵值（REV）派生而来。每笔交易的授权使用轻量级基于HMAC的证明（CPU上每笔交易约1微秒），而非每笔交易的密码学签名，绑定证明被推迟到非关键路径的零知识证明[3]中处理。

这种分离对VM是无感知的：身份承诺和证明机制独立于任何特定执行环境。因此，只要存在从 id_com 到每个VM原生地址格式的确定性、抗碰撞映射，单个 id_com 就可以作为跨任意多个VM的地址锚点。

1.3 贡献

本文做出以下贡献：

1. 广义n-VM架构。我们定义了一个在单一Layer-1链上托管n个对等虚拟机的框架，具备基于操作码前缀的交易路由、共享状态树和可插拔引擎接口（第3节）。
2. 统一身份层。我们形式化了从单一身份承诺确定性派生VM特定地址的过程，在HKDF伪随机函数假设下证明了上下文隔离性，并描述了面向旧版钱包的原始链入口验证（第4节）。
3. 统一代币账本。我们提出了一个单账本代币运行时，在相同的底层余额存储上暴露ERC-20和SPL接口，并证明跨VM代币转移是原子的（第5节）。
4. 并行执行。我们描述了基于写集的冲突检测调度器和基于上下文的分片方案，并分析了吞吐量扩展性（第6节）。
5. 具体实例化。我们描述了一个Rust实现，包含n=5个VM（Native、EVM、SVM、BVM、TVM），含身份预编译合约、跨VM调用钩子以及原始链签名验证（第9节）。

n-VM构建于Ace-GF身份框架[1]（多流密钥派生）、Ace Runtime执行层[3]（证明-执行-验证流水线）以及VA-DAR恢复协议[2]（基于邮箱的钱包恢复）之上。

2 背景

2.1 ACE-GF身份原语

原子密码学实体生成框架（Ace-GF）[1]从称为根熵值（REV）的单一高熵秘密中派生确定性、特定用途的密钥流。第i个密钥流为：

$$k_i \leftarrow \text{HKDF-SHA256}(\text{REV}, \text{info}_i, \text{salt}_i) \quad (\text{公式1})$$

Ace-GF为多个密码学生态系统定义了规范流：

- 流1：Ed25519（Solana签名）
- 流3：secp256k1（EVM链）
- 流4：secp256k1（比特币）
- 流7：ML-DSA-44（后量子签名）

链上身份由Poseidon哈希承诺表示：

$$id_com = \text{Poseidon}(\text{REV}, \text{salt}, \text{domain}) \quad (\text{公式2})$$

这个32字节的值是链上可见的主要原生身份工件。在原生Ace-GF路径中，长期公钥的披露可以推迟或避免，从而减少对已发布公钥进行“先收割后解密”攻击的暴露风险。

2.2 证明-执行-验证流水线

Ace Runtime[3]通过三阶段流水线处理交易：

1. 证明阶段（每笔交易约1-5微秒，CPU）：对每笔交易验证轻量级基于HMAC的证明凭据。
2. 执行阶段（每笔交易约10-50微秒）：针对状态树执行交易。
3. 验证阶段（非关键路径，GPU）：生成将所有证明绑定到其各自id_com值的零知识证明。

第1阶段和第2阶段位于关键路径上，决定出块时间（约400毫秒）。第3阶段异步运行，在后续时隙中产生密码学最终性证书。n-VM调度器将单体执行阶段替换为一个路由层，将任务委托给n个可插拔VM引擎。

2.3 现有多VM方案

我们简要综述现有多VM设计并指出其局限性。

表1：多VM方案比较。

系统	虚拟机	身份	代币转移
Movement	EVM + Move	各VM独立	桥接
Eclipse	以太坊上的SVM	各VM独立	桥接（L1↔L2）
Sei v2	EVM + CosmWasm	指针合约	CosmWasm↔EVM指针
Polkadot	各平行链独立	各平行链独立	XCM消息传递
n-VM（本工作）	n个对等VM	单一id_com	统一账本（无桥接）

关键区别在于：n-VM将所有VM视为共享单一共识、身份和代币层的一等公民，而非使一个VM从属于另一个VM。

3 架构

3.1 概览

n-VM架构由四个层次组成：

共识层（BFT + 证明-执行-验证）

n-VM调度器（操作码路由）

VM1（原生） VM2（EVM） VM3（SVM） · · · VMn

统一状态树 + 身份层 + 代币账本

图1：n-VM分层架构。

3.2 基于操作码的交易路由

每笔交易携带一个载荷，其第一个字节决定目标VM。我们将256个操作码空间划分为n个连续区间：

定义1（操作码路由函数）。设 $V = \{V_1, V_2, \dots, V_n\}$

为已注册VM的集合。每个 V_i 被分配一个连续操作码区间 $[l_i, u_i] \in [0,$

255]，各区间两两不相交。路由函数为：

$\text{Route}(\text{opcode}) = V_i$ ，若 $l_i \leq \text{opcode} \leq u_i$ ；否则为 $\perp$ （拒绝交易）。

该设计具有以下优点：

- O(1)路由。单字节比较即可确定目标VM，对交易处理几乎无额外开销。
- 可扩展性。通过为未占用的操作码区间注册引擎即可添加新VM，无需修改现有引擎。
- 确定性。路由决策仅取决于交易载荷，确保所有验证者得出相同的调度决策。

3.3 VM引擎接口

每个VM引擎实现一个最小接口：

定义2（VM引擎）。VM V_i 的引擎 E_i 实现：

1. $vm_id() \rightarrow V_i$ ：返回VM标识符。
2. $execute(state, tx) \rightarrow (receipt, \Delta)$ ：针对状态树state执行交易tx，返回回执和状态变更集 Δ 。

调度器维护一个注册表 $E: V \rightarrow E$ ，将每个VM标识符映射到其引擎。交易执行遵循算法1。

算法1：n-VM交易调度

输入：交易tx（含载荷P）、状态树S

输出：回执R

- 1: $opcode \leftarrow P[0]$
- 2: $V \leftarrow Route(opcode)$
- 3: 若 $V = \perp$ 则返回 Reject("未知操作码")
- 4: $E \leftarrow E[V]$
- 5: $S' \leftarrow Snapshot(S)$ 保存状态以备回滚
- 6: $(R, \Delta) \leftarrow E.execute(S, tx)$
- 7: 若 $R.success = false$ 则 Rollback(S, S') 恢复执行前状态
- 8: 返回 R

快照/回滚机制确保任何VM中失败的交易都不会破坏共享状态树，从而保持VM间的隔离性。

3.4 区块执行

区块 $B = [tx_1, tx_2, \dots, tx_m]$ 是可能针对不同VM的有序交易序列。调度器（在基准模式下）顺序处理每笔交易，原子地应用状态变更：

$$S_j = S_{j-1} \cup \Delta_j \text{ (若 } tx_j \text{ 成功)}, \text{ 否则 } S_j = S_{j-1}, j = 1, \dots, m \text{ (公式3)}$$

给定初始状态 S_0 和区块B，最终状态 S_m 及有序回执列表 $[R_1, \dots, R_m]$ 是确定性的。

4 统一身份层

4.1 跨VM地址派生

核心身份原语是从单一id_com到VM特定地址的确定性映射。

定义3（地址派生）。对于原生地址长度为 l_i 字节的VM V_i ，派生地址为：

$$a_{Vi} = \text{Truncate}_{l_i}(\text{SHA-256}(\text{tag}_i || \text{id_com}))$$

其中 tag_i 是VM特定的域分隔符字符串， Truncate_{l_i} 取哈希输出的最后 l_i 个字节。

对于 $n=5$ 的实例化：

$$a_{EVM} = \text{SHA-256}(\text{"evm:"} || \text{id_com})[12:32] \text{ (20字节)} \text{ (公式4)}$$

$a_{SVM} = \text{SHA-256}(\text{"svm:"} || \text{id_com})$ (32字节) (公式5)

$a_{BVM} = \text{SHA-256}(\text{"bvm:"} || \text{id_com})$ (32字节) (公式6)

$a_{TVM} = \text{SHA-256}(\text{"tron:"} || \text{id_com})[12:32]$ (20字节) (公式7)

$a_{\text{native}} = \text{id_com}$ (32字节) (公式8)

定理4.1 (地址隔离)。对于任意两个不同的VM标签 $\text{tag}_i \neq \text{tag}_j$ 和任意 id_com ，派生地址 a_{vi} 和 a_{vj} 在SHA-256碰撞抗性假设下是计算独立的。

证明概要。SHA-256被建模为随机预言机。由于 $\text{tag}_i \neq \text{tag}_j$ ，输入 $\text{tag}_i || \text{id_com}$ 和 $\text{tag}_j || \text{id_com}$ 是不同的，因此其输出是独立分布的。截断保持计算独立性：知道截断输出无助于预测不同哈希的不同截断。这意味着用户在某一VM上的地址被泄露，不会透露其与其他VM上的地址信息——除了通过共享 id_com 已公开的部分。

4.2 反向解析

为了与旧版工具（例如显示EVM地址的区块浏览器）的互操作性，调度器维护一个反向索引：

$\text{ace_id_from_evm}(a_{\text{EVM}}) = \text{SHA-256}(\text{"ace_from_evm:"} || a_{\text{EVM}})$ (公式9)

这是一个确定性的单向映射，允许EVM合约通过预编译合约查询EVM地址对应的ACE侧身份，而不暴露原始 id_com 。

4.3 原始链入口

为实现冷启动采用，n-VM链必须接受由旧版钱包（EVM的MetaMask、Solana的Phantom等）签名的交易，这些钱包不了解Ace-GF证明机制。

定义4（原始链交易）。原始链交易是除标准载荷外还携带原生链签名（EVM的ECDSA、Solana的Ed25519等）的交易。调度器执行：

1. 签名验证：使用链原生验证算法恢复签名者的公钥或地址。
2. 身份映射：为恢复的地址计算确定性的 id_com 。
3. 载荷重建：验证交易载荷与从原始字节派生的规范形式匹配。
4. 账户绑定：确保派生的 id_com 在状态树中有账户，必要时创建一个。

该机制支持四种入口路径：

表2：原始链入口验证。

链	签名	地址格式	身份派生
EVM	ECDSA/secp256k1	20字节Keccak	$\text{SHA-256}(\text{"legacy_evm:"} \text{addr})$
Solana	Ed25519	32字节公钥	$\text{SHA-256}(\text{"legacy_sol:"} \text{pubkey})$
Bitcoin	ECDSA/Schnorr	P2WPKH/Taproot	$\text{SHA-256}(\text{"legacy_btc:"} \text{pubkey})$
Tron	ECDSA/secp256k1	20字节 (T前缀)	$\text{SHA-256}(\text{"legacy_tron:"} \text{addr})$

原始链入口提供了无缝迁移路径：用户可以先用现有钱包与n-VM链交互，然后选择性升级为Ace-GF身份以实现跨VM统一和基于证明的授权。

5 统一代币账本

5.1 设计

代币标准在各VM间的碎片化是多链系统复杂性的主要来源之一。EVM使用ERC-20（带授权额度的余额映射）；SVM使用SPL代币（带铸造者/所有者元数据的独立代币账户）。这些在语义上等价，但在结构上不兼容。

n-VM代币账本将所有代币状态存储在状态树内的单一内置程序账户中，并将VM原生接口作为该共享存储的视图暴露出来：

定义5（统一代币状态）。对于铸造标识符为M（32字节）的代币，规范状态包含：

- $\text{supply}(M)$ ：总供应量（uint64）
- $\text{decimals}(M)$ ：小数精度（uint8）
- $\text{authority}(M)$ ：铸造权限id_com（32字节）
- $\text{balance}(M, \text{id_com})$ ：身份余额（uint64）
- $\text{allowance}(M, \text{id_com}_{\text{owner}}, \text{id_com}_{\text{spender}})$ ：委托授权额度（uint64）

存储槽通过域分隔哈希派生：

$$\text{slot}_{\text{balance}} = \text{SHA-256}(\text{"balance:"} \parallel M \parallel \text{id_com}) \quad (\text{公式10})$$

$$\text{slot}_{\text{allowance}} = \text{SHA-256}(\text{"allowance:"} \parallel M \parallel \text{id_com}_{\text{owner}} \parallel \text{id_com}_{\text{spender}}) \quad (\text{公式11})$$

5.2 多接口访问

相同的底层余额可通过EVM和SVM接口访问：

ERC-20接口。每个铸造者M有一个确定性的ERC-20合约地址：

$$\text{addr}_{\text{ERC20}} = \text{SHA-256}(\text{"erc20-addr:"} \parallel M)[12:32] \quad (\text{公式12})$$

当EVM合约在该地址调用 $\text{balanceOf}(a_{\text{EVM}})$ 时，运行时：（1）通过反向索引将 a_{EVM} 解析为对应的id_com；（2）从统一账本读取 $\text{balance}(M, \text{id_com})$ ；（3）以ERC-20兼容的ABI编码返回结果。

SPL接口。SPL代币账户被建模为兼容性别名。关联代币地址（ATA）使用标准Solana

PDA算法派生：

$$\text{ata} = \text{FindPDA}([\text{owner_pubkey}, \text{spl_program_id}, M], \text{ata_program_id}) \quad (\text{公式13})$$

当SVM程序查询代币账户余额时，运行时将ATA解析为所有者的id_com，并从相同的统一账本读取数据。

5.3 跨VM转移原子性

定理5.1（跨VM原子性）。从EVM上下文到SVM上下文的代币转移作为统一账本上的单一状态转换执行，不需要任何中间桥接状态。

证明概要。EVM和SVM接口都解析为对以id_com为键的相同存储槽的操作。从id_com_A到id_com_B的转移恰好修改两个槽：

$$\text{balance}(M, \text{id_com}_A) -= v, \text{balance}(M, \text{id_com}_B) += v$$

由于两次写操作都发生在单一状态树上的单笔交易中，根据公式3的交易执行语义，它们是原子的。不存在中间“锁定”或“飞行中”状态。这消除了一整类桥接相关漏洞：不存在锁定-铸造-销毁-释放循环，没有多签委员会，也没有乐观挑战期。

6 并行执行

6.1 写集冲突检测

为利用异构VM工作负载（例如，针对不相交账户的EVM合约调用和原生转账）的内在并行性，n-VM调度器从每笔交易中提取写集并构建无冲突批次。

定义6（写集）。交易tx的写集 $W(tx)$ 为：

$W(tx) = \{a_1, a_2, \dots\}$ （若写入账户可静态确定），否则为（全局，与所有交易冲突）

对于写集可静态确定的交易（原生转账、SVM转账、BVM转账），调度器直接从载荷中提取发送方和接收方账户ID。对于具有无限副作用的交易（EVM CALL、EVM CREATE、SVM INVOKE），写集被保守地标记为全局。

定义7（冲突）。两笔交易 tx_i 和 tx_j 冲突，当且仅当：

$W(tx_i) = W(tx_j)$ 或 $W(tx_i) \cap W(tx_j) \neq \emptyset$

算法2：批次构造

输入：有序交易列表 $[tx_1, \dots, tx_m]$

输出：有序无冲突批次列表 $[B_1, B_2, \dots]$

1: $batches \leftarrow []$; $current \leftarrow []$; $used \leftarrow \emptyset$

2: 对 $j = 1, \dots, m$:

3: $w \leftarrow W(tx_j)$

4: 若 $w = \emptyset$ 或 $w \cap used = \emptyset$:

5: 若 $|current| > 0$: 将 $current$ 追加到 $batches$

6: 将 $[tx_j]$ 追加到 $batches$ 全局交易单独运行

7: $current \leftarrow []$; $used \leftarrow \emptyset$

8: 否则:

9: 将 tx_j 追加到 $current$

10: $used \leftarrow used \cup w$

11: 若 $|current| > 0$: 将 $current$ 追加到 $batches$

12: 返回 $batches$

同一批次内的交易并行执行（实现中使用Rayon[13]）；批次之间顺序执行以保持区块级确定性。

6.2 基于上下文的分片

为进一步提高并行性，n-VM调度器支持基于上下文的分片：交易携带一个可选的16字节上下文标签，用于确定其分片分配。

定义8（分片路由）。给定K个分片（默认 $K=64$ ）、VM前缀字符串 vm 和上下文标签 ctx ：

$shard = SHA-256(len(vm) || vm || ctx) \bmod K$ （公式14）

长度前缀防止了例如("evm", "foo:bar")和("evm:foo", "bar")之间的碰撞歧义。针对不同分片的交易独立执行。一个保留的共享分片处理跨越多个分片账户的跨上下文交易，以串行方式执行以确保原子性。

6.3 吞吐量分析

我们在三种执行策略下建模吞吐量。

表3：不同执行策略下的预测吞吐量。

策略	描述	预测TPS
顺序执行	单线程，所有VM	约5,000
并行批处理	写集冲突检测，Rayon	约16,000
上下文分片	64分片+跨上下文检测	约33,000–66,000

吞吐量预测基于以下假设：

- 400毫秒出块时间（与Ace Runtime流水线[3]一致）。
- 每笔交易约1–5微秒的证明验证。
- 每笔交易约10–50微秒的执行时间（取决于VM类型）。
- 16核普通服务器硬件。
- 70%的交易具有可静态确定的写集（原生和简单转账）。

约66,000 TPS的上限假设高分片局部性（大多数交易为分片内交易）和最少的跨上下文流量。

7 跨VM通信

7.1 EVM预编译合约

为使EVM智能合约能与n-VM身份和跨VM层交互，运行时在保留地址处暴露了一组预编译合约：

表4：ACE EVM预编译合约。

地址	名称	功能
0x0100	id_com_verify	验证身份承诺证明
0x0101	context_derive	从输入派生上下文
0x0102	admin_factor_check	检查管理员因子凭据
0x0103	zkace_batch_verify	批量验证ZK-ACE证明
0x0104	multisig_derive	派生多签地址
0x0105	multisig_verify	验证多签证明
0x0106	cross_vm_call	编码EVM发起的跨VM调用请求
0x0107	resolve_svm_addr	解析id_com → SVM地址

cross_vm_call预编译合约（0x0106）提供了一个面向EVM的接口，通过提供程序ID和指令数据来表达针对SVM的调用请求。在当前设计中，该预编译合约定义了跨VM请求的调用信封和调度器路由钩子。具有共享gas计量的完全通用同步返回值路径留作未来工作。

7.2 SVM系统调用

对称地，SVM程序可以通过自定义系统调用访问身份层和互补的跨VM路由钩子：

- ace_attest：在SVM程序中验证证明凭据。
- ace_id_com：查询当前交易发送者的id_com。

- `ace_cross_vm_call`：从SVM程序表达面向EVM的跨VM调用请求。

与EVM预编译合约一起，这些系统调用定义了身份访问和跨VM请求路由的双向接口面。它们规定了多VM交互的编码和调度方式，而完全通用的同步跨VM执行模型仍是未来工作。

8 安全分析

8.1 VM隔离

命题8.1（执行隔离）。VM

V_i 在执行交易tx时发生的故障（回退、gas耗尽、崩溃）不影响任何其他VM

V_j ($j \neq i$) 的状态，也不影响后续执行的任何交易。

证明概要。根据算法1，调度器在委托给任何引擎之前会先保存状态快照。发生故障时，状态被回滚到快照。由于状态树是唯一的共享资源，回滚将其恢复到执行前状态，因此失败交易的任何副作用都不会持续存在。

8.2 身份绑定正确性

命题8.2（原始链绑定）。对于来自链c、恢复地址为addr的原始链交易，到`id_com = SHA-256("legacy_c:" || addr)`的绑定具有：

1. 确定性：相同地址始终映射到相同的`id_com`。
2. 抗碰撞性：两个不同地址以压倒性概率映射到不同的`id_com`值。
3. 域分隔性：即使原始字节相同，不同链的地址也映射到不同的`id_com`值。

证明。性质（1）和（2）直接源于SHA-256是确定性、抗碰撞哈希函数。性质（3）源于链特定前缀：若 $c1 \neq c2$ ，则`"legacy_c1:"` $\neq$ `"legacy_c2:"`，因此即使 $addr1=addr2$ ，SHA-256的输入也是不同的。

8.3 确定性执行

命题8.3（区块级确定性）。给定初始状态 S_0 和区块B，最终状态 S_m 和回执列表 $[R_1, \dots, R_m]$ 在所有验证者之间是相同的。

证明概要。操作码路由函数（定义1）是确定性的。每个VM引擎的`execute`函数是确定性的（`revm`在Shanghai模式下；SVM使用固定内置程序；BVM使用确定性Script评估）。状态快照和回滚是确定性的。批次内的并行执行产生与顺序执行相同的结果，因为同一批次中的交易根据构造（算法2）是无冲突的。上下文分片路由（定义8）是确定性的。

9 实现

9.1 系统概览

n-VM调度器以Rust实现为`ace-n-vm`

crate，包含约5,000行Rust代码和3,000行测试代码。表5总结了具体实例化中的五个VM引擎。

表5：已实现的VM引擎（ $n=5$ ）。

VM	操作码	后端	地址	能力
Native	0x01-0x0F	<code>ace_engine</code>	32字节 <code>id_com</code>	转账、账户创建、认证密钥管理

EVM	0x10-0x1F	revm v19 (Shanghai)	20字节	完整EVM：Solidity、Vyper、ERC-20/721/1155/4626
SVM	0x20-0x2F	内置程序	32字节	SPL代币、SystemProgram、PDA
BVM	0x30-0x3F	Script解释器	32字节	比特币Script、UTXO管理、P2WPKH/Taproot
TVM	0x40-0x4F	revm（重映射）	20字节	通过操作码重映射实现Tron兼容

9.2 TVM作为EVM委托

TVM引擎展示了n-VM架构的可扩展性：TVM不实现独立的执行引擎，而是通过操作码重映射复用EVM引擎：

$$\text{TVM.execute(tx)} = \text{EVM.execute(remap(tx))} \quad (\text{公式15})$$

其中remap将操作码0x40→0x10、0x41→0x11、0x42→0x12进行转换，并将地址命名空间从Tron调整为EVM。该方法以最少代码添加了一个新VM，同时为重映射操作码子集保留了Tron兼容的执行模型。

9.3 原始链验证

每种原始链入口路径实现了链特定的签名恢复和载荷规范化：

- EVM：通过ecrecover恢复secp256k1签名者，验证链ID与证明域匹配，并从原始RLP编码交易重建规范载荷。
- Solana：验证Ed25519签名，提取转账参数，并计算发送者的旧版id_com。Solana重放保护通过每时隙签名去重来强制执行。
- Bitcoin：根据输出类型（P2PKH、P2WPKH、P2WSH、Taproot）验证ECDSA或Schnorr签名，从序列化交易重建规范载荷。
- Tron：类似于EVM，但使用Tron特定的protobuf编码和域槽验证。

10 相关工作

多VM Layer-1链。Sei v2[5]在Cosmos SDK链中融合了EVM和CosmWasm执行，使用指针合约在VM间桥接状态。然而，指针合约引入了一个间接层，无法实现统一身份：EVM和CosmWasm地址仍然是独立的，VM间的代币转移需要显式的指针交互。Artela[6]通过WebAssembly扩展为自定义运行时模块扩展了EVM，但WASM层从属于EVM，而非对等执行环境。

多VM Layer-2系统。Movement Labs[7]在基于Move的结算链之上部署了EVM兼容的执行层。Eclipse[8]将Solana的SVM作为以太坊Layer-2 Rollup运行。这些方案继承了其各自L1/L2结算机制的信任假设和延迟，且各层间的身份仍然碎片化。

跨链互操作性。Polkadot的XCM[9]和Cosmos的IBC[10]支持独立链之间的消息传递，但不提供统一身份或共享状态。每条链维护自己的账户模型，代币转移需要锁定-铸造或销毁-释放桥接机制。

并行执行。Solana的Sealevel[11]通过要求交易预先声明访问账户，开创了账户级并行性。Aptos的Block-STM[12]使用乐观并发控制，在冲突时重新执行。n-VM调度器将写集提取与基于上下文的分片相结合，针对跨VM并行性，而非单一VM的账户级并行性。

11 结论

我们提出了n-VM，一种在单一Layer-1区块链上将n个异构虚拟机作为对等执行引擎托管的广义架构。该设计的三大支柱——基于操作码的调度、通过Ace-GF承诺的统一身份，以及具有多接口访问的单账本代币运行时——共同消除了在不同执行环境中操作时对桥接、封装代币或多个钱包的需求。

该架构以n=5个VM（Native、EVM、SVM、BVM、TVM）实例化，但框架对n是参数化的：添加新VM只需实现引擎接口并注册操作码区间，无需更改身份层、代币账本或现有引擎。TVM引擎通过操作码重映射和命名空间转换复用现有EVM后端，在实践中展示了这种可扩展性。

在第6节的分析假设下，写集冲突检测和基于上下文的分片建议在普通硬件上的潜在吞吐量范围为约16,000–66,000 TPS。

展望未来，几个自然的扩展方向包括：

- WASM虚拟机。添加WebAssembly执行引擎（例如，用于CosmWasm或ink!合约）作为VM6，将把兼容性扩展到Cosmos和Polkadot生态系统。
- Move虚拟机。集成Move执行引擎将提供与Aptos和Sui智能合约的兼容性。
- 投机并行执行。将保守写集调度器替换为乐观并发控制（Block-STM风格）可以提高写集无法静态确定的EVM和SVM交易的并行性。
- 跨VM可组合性。将当前的预编译/系统调用请求钩子推广为具有共享gas计量的同步跨VM调用，将使原子多VM交易成为可能。

参考文献

- [1] J. S. Wang. ACE-GF: A Generative Framework for Atomic Cryptographic Entities. arXiv preprint arXiv:2511.20505, 2025.
- [2] J. S. Wang. VA-DAR: A PQC-Ready, Vendor-Agnostic Deterministic Artifact Resolution for Serverless, Enumeration-Resistant Wallet Recovery. arXiv preprint arXiv:2603.02690, 2026.
- [3] J. S. Wang. ACE Runtime: A ZKP-Native Blockchain Runtime with Sub-Second Cryptographic Finality. arXiv preprint arXiv:2603.10242, 2026.
- [4] Chainalysis. Cross-chain Bridge Hacks. Chainalysis Report, 2024.
- [5] J. Gorzny et al. Sei v2: The First Parallelized EVM. Sei Labs Whitepaper, 2024.
- [6] Artela Network. Artela: EVM++ with Aspect Programming. Artela Whitepaper, 2024.
- [7] Movement Labs. Movement: Modular Framework for Building and Deploying Infrastructure, Applications, and Blockchains Built on Move. Movement Whitepaper, 2024.
- [8] Eclipse. Eclipse: The First SVM Layer 2 on Ethereum. Eclipse Documentation, 2024.
- [9] G. Wood. Polkadot: Vision for a Heterogeneous Multi-Chain Framework. Polkadot Whitepaper, 2016.
- [10] C. Goes et al. The Interblockchain Communication Protocol: An Overview. arXiv preprint arXiv:2006.15918, 2020.
- [11] A. Yakovenko. Solana: A New Architecture for a High Performance Blockchain. Solana Whitepaper, 2018.
- [12] Aptos Labs. Block-STM: Scaling Blockchain Execution by Turning Ordering Curse to a Performance Blessing. arXiv preprint arXiv:2203.06871, 2022.

[13] N. Matsakis and A. Turon. Rayon: Data-parallelism Library for Rust.
<https://github.com/rayon-rs/rayon>, 2015-2026.